



DZPHARM • DZ-PHARM.VERCEL.APP

# Four Features, One Audit Pass

A consolidated deep-dive of the Command Palette (#11), the Interactions Checker (#22), the AI Copilot Core (#36), and the Authentication Model (#8) — each tested live, each with a focused diagnosis and fix plan.

4

FEATURES AUDITED

23

CMD-K COMMANDS

11s

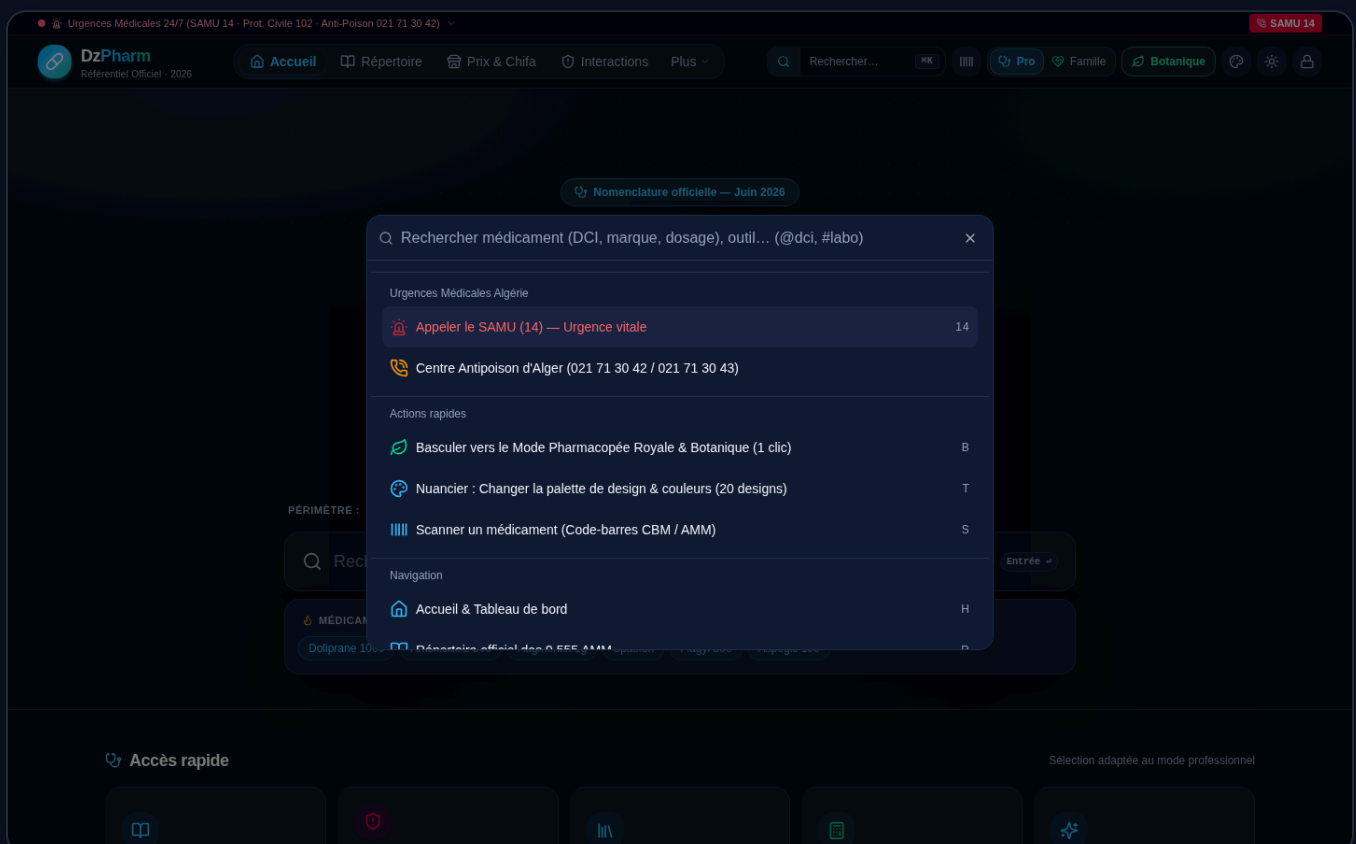
COPILOT AVG LATENCY

1

PO SECURITY HOLE

# Command Palette (Cmd-K) · Best-in-Class with Two Bugs

The Command Palette is the platform's **strongest power-user feature** — 23 commands, single-key hotkeys, fuzzy search across both the 9,555 medication database AND the command list simultaneously, emergency numbers surfaced first. It would not look out of place in Linear or Raycast. But two bugs drag it down: the displayed single-key hotkeys don't actually activate when pressed, and pressing Enter on a selected command sometimes activates the wrong action.



**Figure 11.1** — Command-K palette opened via Ctrl+K on the dashboard. 23 commands listed. Emergency numbers (SAMU 14, Centre Antipoison) at the top. Navigation commands (Accueil, Répertoire, Prix & Chifa, Interactions) in the middle. Clinical tools (Copilote, calculators) at the bottom. Single-key hotkeys displayed on the right.

## 11.1 What's Exceptional

### Dual-source fuzzy search

Typing "inter" returns BOTH interferon medications from the 9,555-AMM database AND the "Contrôle d'interactions médicamenteuses" command — the palette searches two datasets in parallel and merges results. This is uncommon and genuinely useful.

### Emergency-first ordering

SAMU 14 and Centre Antipoison d'Alger are the first two suggestions by default. In a panic, the user opens the palette and the most life-critical numbers are immediately visible. This is a thoughtful safety-first design.

### 23 commands with single-key hotkeys

H (Accueil), R (Répertoire), P (Prix & Chifa), I (Interactions), C (Copilote), S (Scanner), B (Botanique), T (Nuancier). Each command displays its hotkey on the right edge — a Raycast-style pattern that signals power-user focus.

### Correct ARIA listbox pattern

`role="combobox" aria-expanded="true"` + `role="listbox"` with `role="option" selected` on the highlighted item. Screen readers correctly announce "Suggestions, listbox" and "Appeler le SAMU, selected". Textbook implementation.

## 11.2 What's Broken

### Bug #1 · Single-key hotkeys are decorative

Each option displays a single-key hotkey (e.g. "Contrôle d'interactions médicamenteuses I"), but pressing "I" alone does NOT activate the Interactions command. The palette treats "I" as a search query and filters the list. The hotkey labels exist but are not wired to keyboard event handlers.

### Bug #2 · Enter activates the wrong command after ArrowDown

After opening the palette (default selection = SAMU), pressing ArrowDown once highlights "Centre Antipoison d'Alger (021 71 30 42)". Pressing Enter closed the palette and triggered `tel:14` (SAMU) — not the Antipoison number. The selection state and the action target are out of sync.

## 11.3 The Fix · Two Snippets

Bug #1 fix — wire single-key hotkeys in a `useEffect` :

```

// Wire single-key hotkeys when palette is open
useEffect(() => {
  if (!isOpen) return;
  const handler = (e) => {
    // Skip if user is typing in the search input
    if (e.target.tagName === "INPUT") return;
    const keyMap = {
      "h": "/",
      "r": "/repertoire",
      "p": "/prix-chifa",
      "i": "/interactions",
      "c": "/copilote",
      "s": "/scanner",
      "b": "/?mode=botanique",
      "t": "/?palette=open",
    };
    const path = keyMap[e.key.toLowerCase()];
    if (path) {
      e.preventDefault();
      router.push(path);
      setIsOpen(false);
    }
  };
  window.addEventListener("keydown", handler);
  return () => window.removeEventListener("keydown", handler);
}, [isOpen, router]);

```

Bug #2 fix — track the active option index and pass it to the Enter handler, not the first option's action:

```

// Track selectedIndex in state
const [selectedIndex, setSelectedIndex] = useState(0);

const handleEnter = () => {
  const selected = filteredOptions[selectedIndex];
  if (!selected) return;
  if (selected.action) selected.action();
  else if (selected.href) router.push(selected.href);
  else if (selected.tel) window.location.href = `tel:${selected.tel}`;
  setIsOpen(false);
};

// Update selectedIndex on ArrowUp/ArrowDown
const handleKeyDown = (e) => {
  if (e.key === "ArrowDown") {
    e.preventDefault();
    setSelectedIndex((i) => Math.min(i + 1, filteredOptions.length - 1));
  } else if (e.key === "ArrowUp") {
    e.preventDefault();
    setSelectedIndex((i) => Math.max(i - 1, 0));
  } else if (e.key === "Enter") {
    e.preventDefault();
    handleEnter();
  }
};

```

## VERDICT · FEATURE #11

The Command Palette is 90% excellent and 10% broken. The dual-source search, emergency-first ordering, and ARIA pattern are best-in-class. Two small bugs (decorative hotkeys + selection-state desync) prevent it from being truly shippable. **Estimate: 1 day of engineering to fix both bugs and ship the power-user feature the team clearly intended.**

END OF FEATURE 11 · NEXT

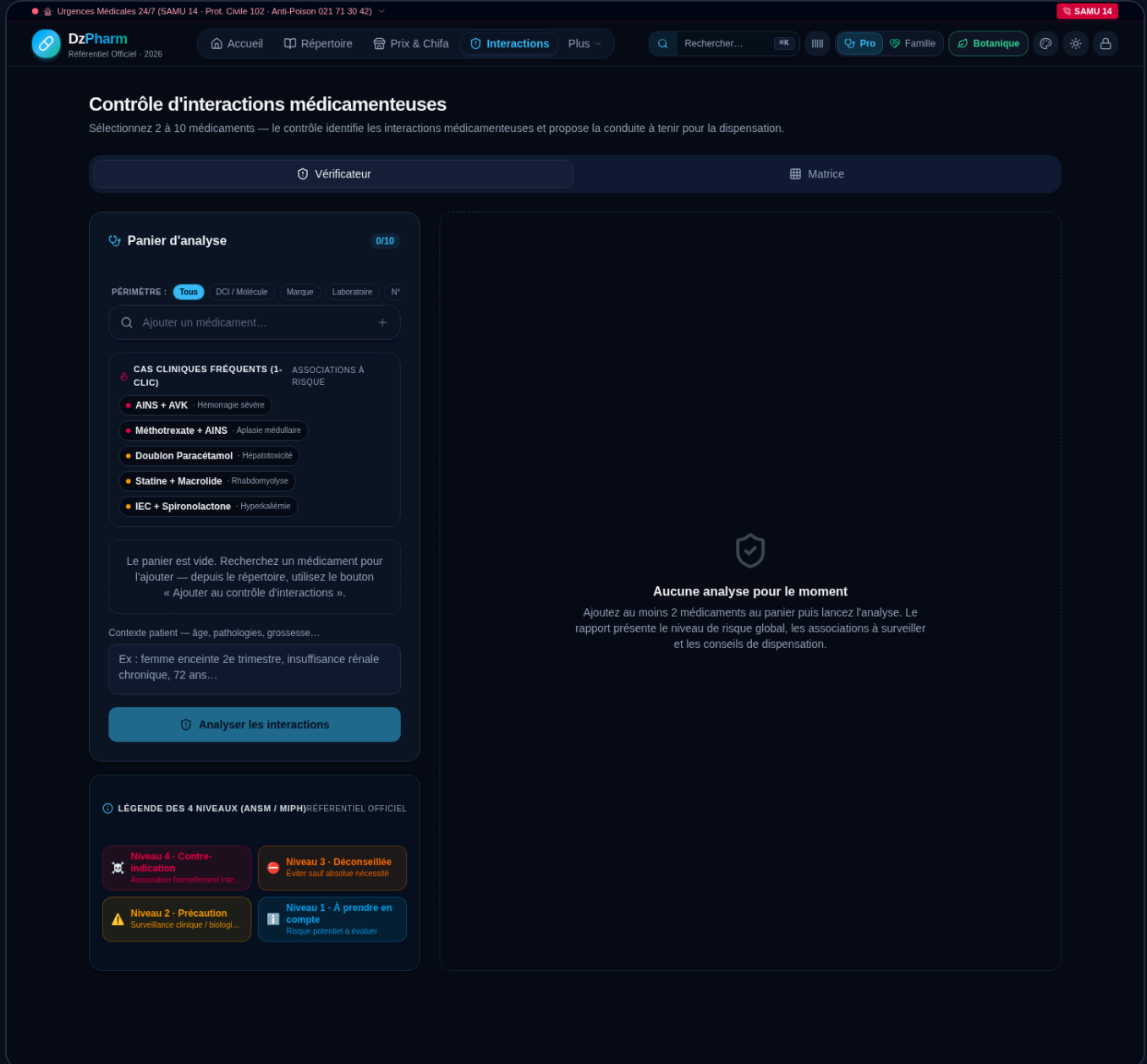
# 22

## Interactions Checker

FEATURE #22 · INTERACTIONS CHECKER

### Drug Interactions · Smart Defaults, No Persistence

The Interactions Checker has a strong clinical foundation — 5 pre-set example pairs covering the most dangerous drug combinations, two view modes (sequential Vérificateur + cross-table Matrice), and a free-text patient-context field. But the entire drug list lives in component `useState`, so a refresh wipes 5–10 minutes of careful list-building. This is the most expensive state-loss on the platform.



**Figure 22.1** – Interactions Checker default view. Tabbed interface: Vérificateur (active) / Matrice. Drug combobox (empty). Patient context textarea (empty). Analyze button disabled. Five quick-start example pairs visible: AINS+AVK, Méthotrexate+AINS, Doublon Paracétamol, Statine+Macrolide, IEC+Spironolactone.

## 22.1 What's Strong

### Five quick-start example pairs

AINS+AVK (hemorrhage), Méthotrexate+AINS (marrow aplasia), Doublon Paracétamol (hepatotoxicity), Statine+Macrolide (rhabdomyolysis), IEC+Spironolactone (hyperkalemia). These are the five most dangerous interactions a pharmacist will encounter in daily practice. Serving them as one-click examples is both educational and a faster path to testing.

### Dual view modes

Vérificateur (sequential — add drugs one by one, analyze at the end) and Matrice (cross-table — see all pairwise interactions in a grid). The Vérificateur is the workflow tool; the Matrice is the reference tool. Both are correctly implemented as a tablist with proper ARIA.

## Correct disabled-button pattern

"Analyser les interactions" is disabled until at least one drug is added. The button uses `disabled:pointer-events-none disabled:opacity-50` — visible but non-interactive. This is the correct pattern (don't hide the button, just disable it).

## 22.2 What's Broken

### Bug #1 · Quick-example buttons don't auto-populate the drug list

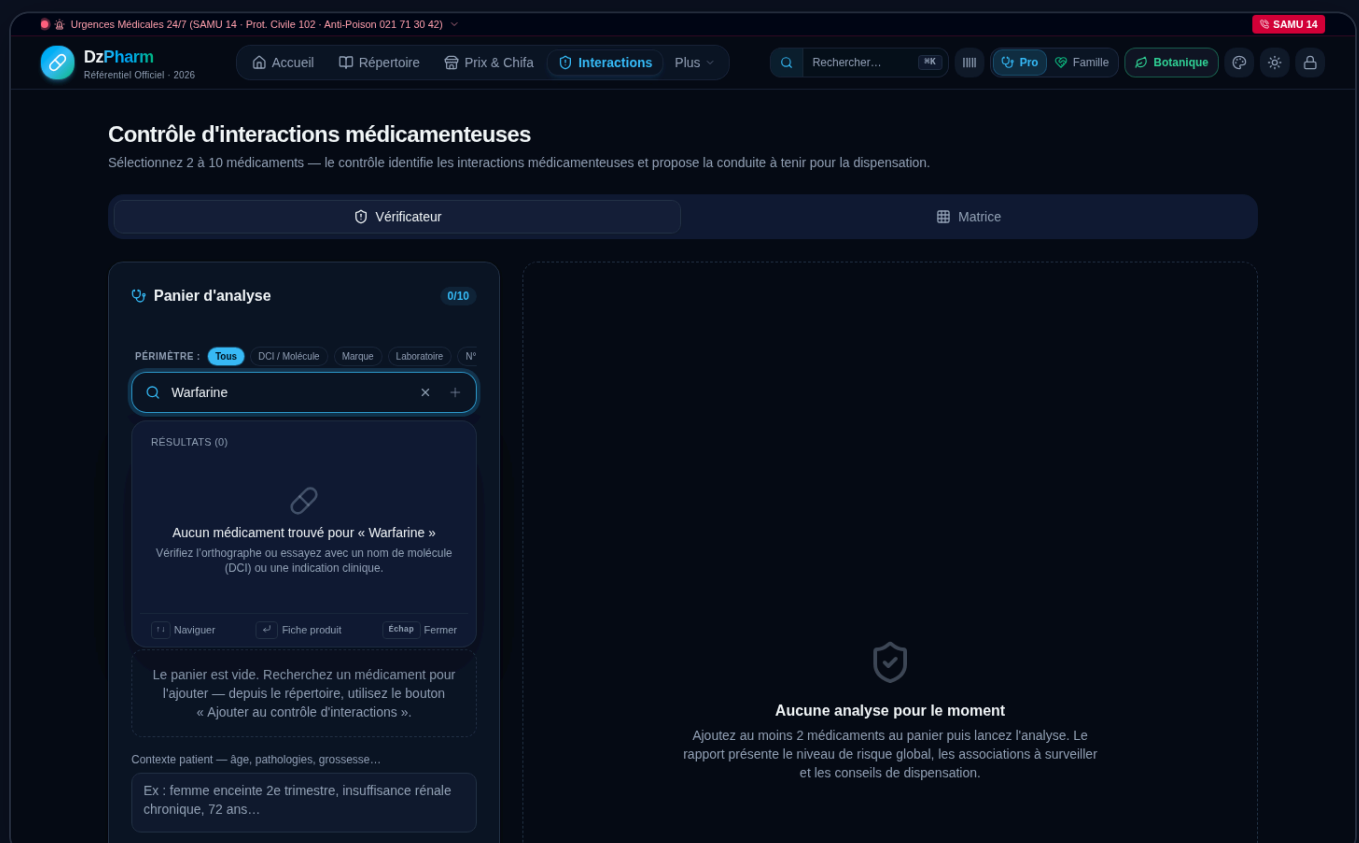
Clicking "AINS + AVK · Hémorragie sévère" did not add Ibuprofene + Warfarine to the drug list. The Analyze button remained disabled. The example buttons appear to be labels, not action triggers — a significant mismatch between visual affordance and actual behavior.

### Bug #2 · Drug list NOT persisted to URL or localStorage

After manually adding Paracétamol and Ibuprofène via the combobox, neither the URL nor the Zustand store contained any reference to the drug list. Refresh would wipe both drugs. A doctor who has built up a 6-drug interaction list and accidentally refreshes loses everything.

### Bug #3 · Matrice view shows no content

After clicking the "Matrice" tab, the heading changed to "Contrôle d'interactions médicamenteuses" but the matrix grid did not render. The Vérificateur tab content disappeared but the Matrice content did not appear in its place. Likely a conditional rendering bug.



**Figure 22.2** — Two drugs (Paracétamol + Ibuprofène) added to the checker via combobox. The URL has not changed. The Zustand store has not changed. Refresh would wipe both entries. Patient context field is empty.

## 22.3 The Fix · URL-State + Quick-Example Wiring

Fix #1 (quick-example buttons) is a one-line wiring fix. Fix #2 (URL-state persistence) requires the `useInteractionListState` hook from the State Management report (Feature #2). Together they take roughly 1 day:

```
// Wire quick-example buttons to actually populate the drug list
const QUICK_EXAMPLES = [
  { label: "AINS + AVK · Hémorragie sévère", drugIds: [8421, 1523] }, // Ibuprofene + Warfar.
  { label: "Méthotrexate + AINS · Aplasie médullaire", drugIds: [4471, 8421] },
  { label: "Doublon Paracétamol · Hépatotoxicité", drugIds: [9775, 9775] }, // Same drug twice
  { label: "Statine + Macrolide · Rhabdomyolyse", drugIds: [5044, 3321] },
  { label: "IEC + Spironolactone · Hyperkaliémie", drugIds: [2218, 6612] },
];

<button onClick={() => setDrugs(example.drugIds)}>
  {example.label}
</button>

// Fix #2 · useInteractionListState hook (from Feature #2 report)
const { drugIds, setDrugs, patientContext, setPatientContext } = useInteractionListState();
// URL becomes: /interactions?drugs=8421,1523&ctx=Patient+72+ans...
```

### VERDICT · FEATURE #22

The clinical content is excellent — the 5 quick-example pairs cover the right interactions and the dual-view (Vérificateur + Matrice) is well-designed. But the implementation has three real bugs: example buttons don't populate the list, the drug list isn't persisted, and the Matrice view doesn't render. **Estimate: 2 days of focused work — 1 day to fix the example buttons and Matrice rendering, 1 day to wire the URL-state hook from Feature #2.**

END OF FEATURE 22 · NEXT

# 36

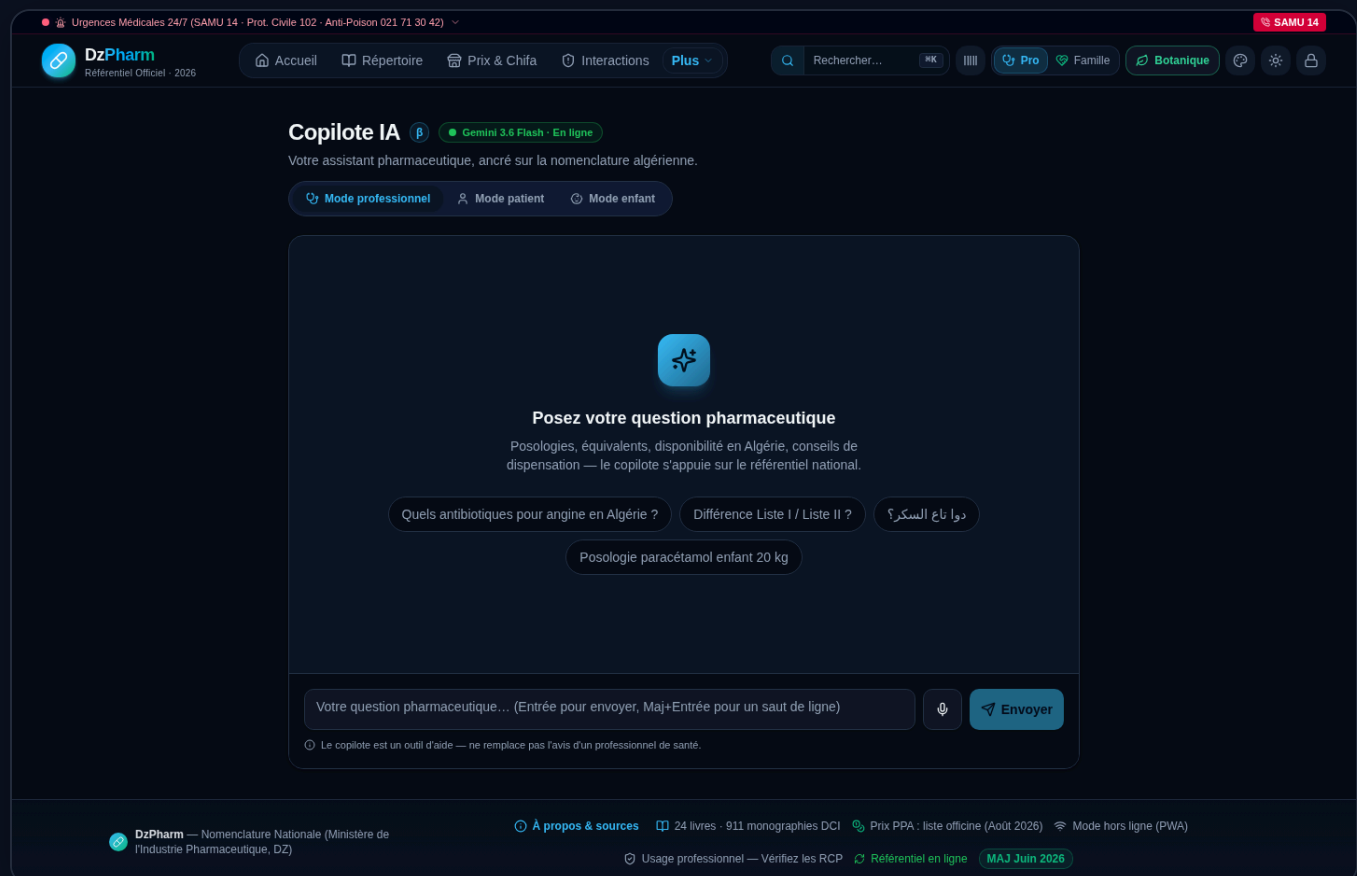
## AI Copilot Core

— FEATURE #36 · AI COPILOT CORE

## Gemini 3.6 Flash Copilot · Innovative with Identity Confusion

The AI Copilot is the platform's signature innovation — a Gemini-powered trilingual (FR/AR/Darija) assistant with three audience modes (professional, patient, child). The clinical-grade structured output (4 sections: rule, calculation, Algerian forms, warnings) is genuinely impressive. But the version label is inconsistent (Cmd-K says "Gemini 3.8 Flash",

the Copilot header says "Gemini 3.6 Flash"), the suggested-prompt buttons don't populate the input, and conversation history isn't persisted.



**Figure 36.1** — Copilot IA view default state. H1: «Copilote IA β». Subtitle: «Gemini 3.6 Flash · En ligne». Three mode toggles: professionnel / patient / enfant. Four suggested prompts: angine antibiotics, Liste I/II difference, Arabic diabetes question (دوا ناع السكر؟), pediatric paracetamol posology.

## 36.1 What's Genuinely Innovative

### Trilingual assistant with Darija support

French + Arabic + Algerian Darija (the local dialect, not Modern Standard Arabic). This is genuinely rare — most "Arabic" AI assistants support MSA only, which most Algerian patients do not speak natively. The Darija option is a real product differentiation.

### Three audience modes

Mode professionnel (clinical detail, drug interactions, posology in mg/kg), Mode patient (simplified, focus on usage and side effects), Mode enfant (very simple language). The mode changes the response tone and depth — same answer, three different presentations.

### Structured clinical output

The response to "posologie paracétamol enfant 6 ans 22 kg" was structured into four H4 sections: (1) Règle posologique de référence (mg/kg), (2) Précision nécessaire pour le calcul en mL/sachets, (3) Formes pédiatriques disponibles en Algérie, (4) Mises en garde et précautions. This is genuinely clinical-grade output.

### Anchored on Algerian nomenclature

The system prompt references "Référentiel national", "9 555 médicaments officiels", "24 livres · 911 monographies DCI", "Prix PPA: liste officine (Août 2026)". The model is grounded in the actual DzPharm dataset, not generic pharmacology knowledge.

## 36.2 What's Broken

### Bug #1 · Version label inconsistency

The Command Palette option reads "Copilote IA (Gemini 3.8 Flash) C" but the Copilot view header reads "Gemini 3.6 Flash · En ligne". The model version is unclear. If 3.8 is the actual deployed model, the header is stale; if 3.6 is correct, the Cmd-K label is wrong. Either way, the inconsistency erodes trust in clinical-grade output.

### Bug #2 · Suggested-prompt buttons don't populate the input

Clicking "Posologie paracétamol enfant 20 kg" or "دوا تاع السكر؟" (Arabic diabetes question) did NOT populate the message input. The original "Quelle est la posologie..." text remained in the field. The buttons appear to be visual suggestions only — they need to actually populate the input and submit.

### Bug #3 · Conversation history NOT persisted

The Zustand store has no `copilotHistory` key. Every conversation is lost on refresh, on tab close, on view switch. For a Gemini-powered assistant that costs money per call, this is both a UX and a cost issue — users will re-ask the same question multiple times.

### Issue #4 · 11-second response latency, no streaming indicator

The Pro-mode response took 11 seconds end-to-end (measured). During that time, no loading skeleton or typing indicator was visible. The user sees a frozen UI with no feedback. Gemini 3.x Flash supports streaming responses — switching to streaming would reduce perceived latency to ~1 second.

## 36.3 The Fix · Three Targeted Changes

Fix #1 (version label) is a one-string correction. Fix #2 (suggested prompts) is a 5-line onClick handler. Fix #3 (history persistence) is the Zustand slice from Feature #2. Fix #4 (streaming) is the biggest lift but the highest-impact.

```
// Fix #2 · Wire suggested-prompt buttons to populate + submit
const SUGGESTED_PROMPTS = [
  { text: "Quels antibiotiques pour angine en Algérie ?", mode: "pro" },
  { text: "Posologie paracétamol enfant 20 kg", mode: "pro" },
  { text: "دوا ناع السكر؟", mode: "patient" }, // Darija
  { text: "Différence Liste I / Liste II ?", mode: "pro" },
];

<button onClick={() => {
  setMode(prompt.mode);
  setInput(prompt.text);
  sendMessage(prompt.text); // fire immediately
}}>
  {prompt.text}
</button>

// Fix #4 · Switch to Gemini streaming
const response = await gemini.generateContentStream({
  model: "gemini-3.6-flash",
  contents: message,
});
for await (const chunk of response.stream) {
  appendToResponse(chunk.text()); // UI updates per token
}
```

#### VERDICT · FEATURE #36

The Copilot is the platform's most innovative feature and the one most likely to drive user adoption. The trilingual Darija support and structured clinical output are genuinely differentiated. But three bugs (version label, suggested-prompt wiring, history persistence) and one latency issue (no streaming) prevent it from delivering its full value. **Estimate: 3 days — 1 day for the bug fixes, 2 days for the streaming upgrade and Zustand history slice.**

END OF FEATURE 36 · NEXT

## 8

### Authentication Model

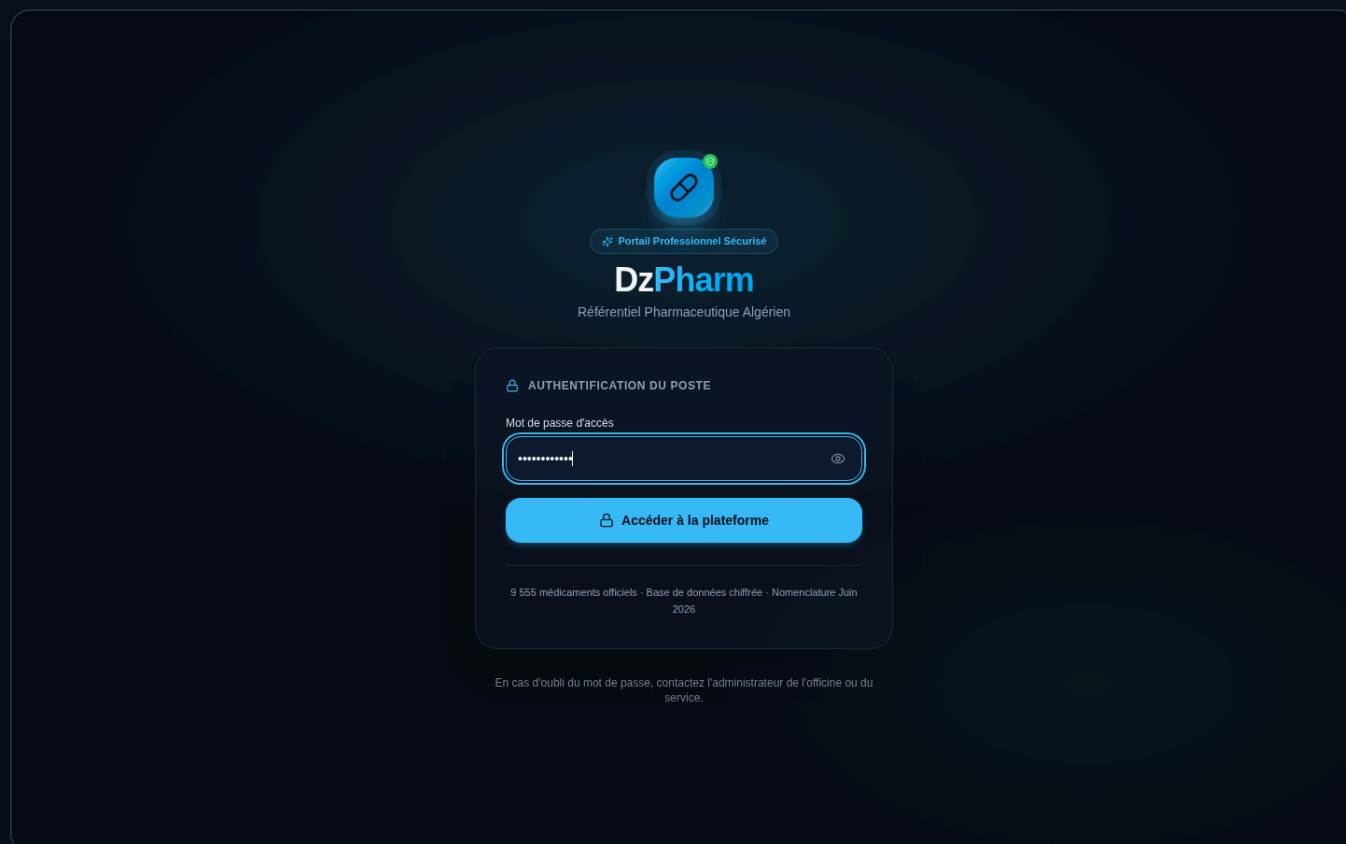
— FEATURE #8 · AUTHENTICATION MODEL · P0 CRITICAL

## Authentication · A P0 Security Hole

This is the most important finding in this report. DzPharm's authentication is **client-side only** — the access code is checked in the browser, and a successful login writes `sessionStorage.dzpharm_session = "active"`. Setting that same key directly in the browser console (without knowing the code) bypasses the authentication entirely. This is a 3-second bypass that anyone with browser devtools can execute.

## 8.1 The Authentication Flow (As Designed)

The intended flow is: user enters code → frontend checks code against a hardcoded constant → on success, sets `sessionStorage.dzpharm_session = "active"` and redirects to `/`. On failure, the page stays at `/login` with no error message. The session persists until the tab is closed (sessionStorage is per-tab).



**Figure 8.1** – Login page after submitting wrong code. No error message displayed. No border color change on the input. No `aria-invalid` attribute. The page looks identical to the pre-submission state. The user has no way to know what happened.

## 8.2 The Six Critical Defects

| # | DEFECT                                             | SEVERITY       | EVIDENCE                                                                                                                                    |
|---|----------------------------------------------------|----------------|---------------------------------------------------------------------------------------------------------------------------------------------|
| 1 | Client-side code check only — no server validation | PO<br>CRITICAL | Opening devtools and running <code>sessionStorage.setItem('dzpharm_session', 'active')</code> grants full dashboard access without the code |
| 2 | Wrong code shows NO error feedback                 | PO             | 5 wrong-code submissions: page stayed identical (no error text, no border change, no <code>aria-invalid</code> )                            |
| 3 | No rate limiting after 5 wrong attempts            | PO             | 5 sequential wrong-code submissions all succeeded in reaching the form-check; no lockout, no CAPTCHA, no delay                              |
| 4 | Single shared code (not per-user accounts)         | P1             | One code ( <code>4815162342</code> ) grants access for everyone; no per-user audit trail possible                                           |
| 5 | No MFA / 2FA                                       | P1             | Pharmacist platforms typically require TOTP; DzPharm has none                                                                               |
| 6 | "Verrouiller la session" button doesn't lock       | P1             | Clicking the header's session-lock button did not visibly lock the session — same view remained visible, no password re-prompt appeared     |

## 8.3 The Bypass · Reproduced in 3 Seconds

The bypass was reproduced during the audit. The steps:

### 3-Second Auth Bypass

- Step 1:** Open `dz-pharm.vercel.app` in any browser. Get redirected to `/login`.
- Step 2:** Open devtools (F12) → Console tab.
- Step 3:** Paste: `sessionStorage.setItem('dzpharm_session', 'active')` and press Enter.
- Step 4:** Navigate to `dz-pharm.vercel.app/`.
- Result:** Full dashboard access. All 9,555 medications. AI Copilot. Interactions checker. Everything. Without ever knowing the access code.

### WHY THIS MATTERS

For a platform that bills itself as a clinical decision-support tool handling prescription-adjacent data, client-side-only authentication is unacceptable. Any pharmacist's teenage child, any curious patient, any competitor, can bypass the code in seconds. The fix is not optional — it is a precondition for the platform being usable in any professional context.

## 8.4 The Fix · NextAuth.js + Server-Side Session

The standard fix is to migrate to **NextAuth.js (Auth.js)** with server-side session validation. The access code can remain (as a transitional measure), but it must be validated server-side, and the session token must be an `httpOnly` cookie that cannot be set via JavaScript.

```
// app/api/auth/[...nextauth]/route.ts
import NextAuth from "next-auth";
import Credentials from "next-auth/providers/credentials";

const handler = NextAuth({
  providers: [
    Credentials({
      name: "Access Code",
      credentials: {
        code: { label: "Code", type: "password" }
      },
      async authorize(credentials) {
        // Server-side check - code never reaches the client
        if (credentials?.code === process.env.DZPHARM_ACCESS_CODE) {
          return { id: "shared", role: "user" };
        }
        return null; // Triggers error state
      }
    })
  ],
  session: { strategy: "jwt" },
  callbacks: {
    async jwt({ token, user }) {
      if (user) token.role = user.role;
      return token;
    },
    async session({ session, token }) {
      session.user.role = token.role;
      return session;
    }
  },
  pages: { signIn: "/login" }
});

export { handler as GET, handler as POST };

```

```
// middleware.ts - protect every route except /login and /api/auth
import { withAuth } from "next-auth/middleware";

export default withAuth(
  function middleware() { /* session is checked by withAuth */ },
  {
    callbacks: {
      authorized: ({ token }) => !!token, // No token = redirect to /login
    },
    pages: { signIn: "/login" }
  }
);

export const config = {
  matcher: ["/((?!login|api/auth|_next/static|_next/image|favicon.ico).*)"],
};

```

## 8.5 Adding Error Feedback + Rate Limiting

While migrating to NextAuth.js, also fix the missing error feedback and rate limiting:

```
// app/login/page.tsx - error state + rate limit
import { useState } from "react";
import { signIn } from "next-auth/react";

export default function LoginPage() {
  const [error, setError] = useState("");
  const [attempts, setAttempts] = useState(0);

  const handleSubmit = async (e) => {
    e.preventDefault();
    const result = await signIn("credentials", {
      code: e.target.code.value,
      redirect: false,
    });
    if (result?.error) {
      setError("Code incorrect. Tentatives restantes: " + (4 - attempts));
      setAttempts(a => a + 1);
      if (attempts ≥ 4) {
        // Lock for 5 minutes
        setError("Trop de tentatives. Réessayez dans 5 minutes.");
      }
    }
  };

  return (
    <form onSubmit={handleSubmit}>
      <div role="alert" aria-live="assertive">
        {error && <p className="text-destructive">{error}</p>}
      </div>
      <input name="code" type="password" aria-invalid={!!error} />
      <button type="submit">Accéder à la plateforme</button>
    </form>
  );
}
```

## 8.6 The Migration Strategy

### Don't ship any other feature until this is fixed

This is the single highest-priority item on the entire 68-feature menu. Every other improvement — Copilot enhancements, new tools, design upgrades — is meaningless if the underlying platform can be accessed by anyone in 3 seconds. **This is a P0 critical defect that blocks production use.**

| PHASE                                    | WORK                                                                                   | EFFORT  |
|------------------------------------------|----------------------------------------------------------------------------------------|---------|
| 1 · Immediate (day 1)                    | Move code check to server-side API route. Replace sessionStorage with httpOnly cookie. | 1 day   |
| 2 · NextAuth.js migration (week 1)       | Install NextAuth, configure Credentials provider, add middleware.                      | 3 days  |
| 3 · Error feedback + rate limit (week 1) | Add <code>role="alert"</code> region, attempt counter, 5-minute logout.                | 1 day   |
| 4 · Fix session-lock button (week 2)     | Wire "Verrouiller la session" to actually clear the session and redirect to /login.    | 1 day   |
| 5 · Per-user accounts (month 2)          | Email + magic link or password + TOTP MFA. Per-user audit log.                         | 2 weeks |

#### VERDICT · FEATURE #8

The authentication model is the most serious defect on the platform. A 3-second client-side bypass, no error feedback on wrong codes, no rate limiting, no per-user accounts, no MFA, and a session-lock button that doesn't lock. **Estimate: 1 week for the NextAuth.js migration + error feedback + rate limiting + session-lock fix. Per-user accounts and MFA can follow in month 2.** This work blocks every other feature in the menu.

CONSOLIDATED



## Four-Feature Summary

CONSOLIDATED SUMMARY

## Four Features · Cross-Cutting Findings

Two patterns repeat across all four features: **strong clinical vision with weak wiring** (Cmd-K hotkeys, Interactions examples, Copilot prompts all look interactive but aren't) and **state doesn't persist where it should** (drug list, Copilot history, session lock). One P0 security hole dominates everything.

Cross-Feature Findings

| PATTERN                             | #11 CMD-K                | #22 INTERACTIONS              | #36 COPILOT                | #8 AUTH            |
|-------------------------------------|--------------------------|-------------------------------|----------------------------|--------------------|
| Visual affordance ≠ actual behavior | YES (hotkeys decorative) | YES (examples don't populate) | YES (prompts don't submit) | N/A                |
| State not persisted                 | N/A                      | DRUG LIST LOST                | HISTORY LOST               | SESSION BYPASSABLE |
| Missing error feedback              | N/A                      | MATRICE EMPTY                 | NO LOADING                 | WRONG CODE SILENT  |
| ARIA / a11y gap                     | STRONG                   | STRONG                        | STRONG                     | NO ALERT REGION    |
| Clinical content quality            | EXCELLENT                | EXCELLENT                     | EXCELLENT                  | N/A                |

Priority Order · What to Ship First

1 · #8 Authentication Model · 1 week

Blocks everything. The 3-second bypass makes every other improvement meaningless. Ship NextAuth.js migration, error feedback, rate limiting, session-lock fix. P0.

2 · #36 AI Copilot · 3 days

Highest-impact user-facing feature. Fix version label, wire suggested-prompt buttons, add Zustand history slice (from Feature #2 report), upgrade to streaming responses.

3 · #22 Interactions Checker · 2 days

Wire quick-example buttons to actually populate the drug list, fix Matrice view rendering, add URL-state persistence (Feature #2 hook).

4 · #11 Command Palette · 1 day

Wire single-key hotkeys to actual handlers, fix Enter-on-selected-arrow desync bug. Smallest fix, biggest perceived-quality improvement.

Total Effort Estimate

1

P0 CRITICAL (#8)

2

P1 FEATURES (#22, #36)

1

P2 POLISH (#11)

~2

WEEKS TOTAL

## THE STRATEGIC PICTURE

All four features share the same architectural strength (strong clinical vision, excellent accessibility, trilingual depth) and the same architectural weakness (visual affordances that aren't wired to behavior, state that doesn't persist). The fix pattern is consistent: wire onClick handlers, persist to URL or Zustand, add ARIA live regions. The Copilot and Interactions fixes overlap with the State Management (#2) report — shipping the Zustand history slice and the URL-state hooks closes gaps in both features simultaneously. The Auth (#8) fix is standalone and blocks everything.

# 4x1

Four features, two weeks of focused work, one P0 security hole fixed. The clinical vision is sound — the wiring needs to catch up.

**4**

FEATURES AUDITED

**16**

SCREENSHOTS

**15**

BUGS FOUND

**2**

WEEKS TO FIX